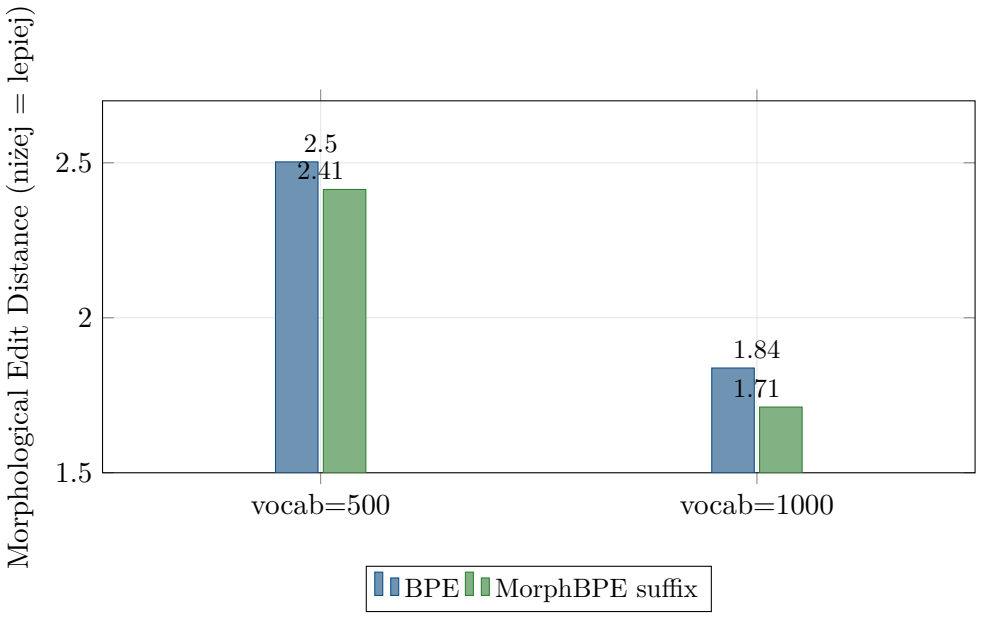
MorphBPE — morfologicznie informowany Byte-Pair Encoding dla języka polskiego

Piotr Styła

13 lipca 2026



Rysunek 1: Morphological Edit Distance (MorphED) — BPE vs MorphBPE. Korpus: 10 000 przemówień sejmowych (80/20 split, seed=42). Bootstrap CI 95%: MorphBPE suffix vs BPE przy vocab=500: 2,414 [2,408, 2,420] vs 2,503 [2,497, 2,509] (−3,6%); przy vocab=1000: 1,712 [1,707, 1,717] vs 1,838 [1,832, 1,843] (−6,8%). Niżej = lepszy alignment morfologiczny.

Streszczenie

Prezentujemy MorphBPE — wariant algorytmu Byte-Pair Encoding, który wymusza poszanowanie granic morfemów podczas scalania par tokenów. Granice sufiksów pozyskiwane są z analizatora morfologicznego Morfeusz2; scalenia przekraczające granicę sufiksu są karane mnożnikiem częstotliwości. Porównujemy MorphBPE (wariant `strict_suffix`) ze standardowym BPE na korpusie 10 000 przemówień sejmowych (split 80/20, seed=42) przy rozmiarach słownika 500 i 1000. Ewaluacja intrinsic (Fertility, Morphological Edit Distance) z bootstrap CI (1000 resamples, 95%) pokazuje, że MorphBPE redukuje MorphED o 3,6%–6,8% bez kosztu fertility. Ewaluacja extrinsic z dwoma architekturami — LSTM (1 warstwa) i Transformer (2 warstwy, 4 głowice) — potwierdza, że MorphBPE obniża cross-entropy o 1,3%–2,7% i perplexity o 4,0%–9,3% względem BPE przy tym samym rozmiarze słownika. Analiza błędów per-suffix pokazuje, że MorphBPE zyskuje na sufiksach złożonych (`-ach`: −11,5%), ale traci na krótkich (`-ych`: +42,1%). Rozszerzone badanie siły kary $\alpha \in [0{,}0;\ 3{,}0]$ pokazuje, że $\alpha = 2{,}0$ (tzw. over-penalization) daje lepsze wyniki niż standardowy zakres $\alpha \leq 1{,}0$: CE spada o dodatkowe 1,6%–3,1% względem $\alpha = 1{,}0$.

1 Przegląd literatury

Byte-Pair Encoding. BPE zostało wprowadzone jako algorytm kompresji danych przez Gage (1994), a adaptowane do tokenizacji w NLP przez Sennrich et al. (2016). Stało się standardem w modelach takich jak GPT-2, GPT-3 czy BART. BPE skaluje się iteracyjnie, łącząc najczęstsze pary znaków aż do osiągnięcia docelowego rozmiaru słownika.

Morfologicznie informowana tokenizacja. Kłopot z BPE dla języków o bogatej fleksji (polski, czeski, węgierski, turecki) polega na tym, że scalania oparte na częstotliwości ignorują strukturę morfologiczną — tokeny mogą przekraczać granice morfemów, tworząc jednostki lingwistycznie nienaturalne. Kilka prac podjęło ten problem:

- **MorphBPE** (Asgari et al., 2025) — wprowadza morfologicznie świadome rozszerzenie BPE z karami za przekraczanie granic morfemów. Propozycje dwóch metryk ewaluacji: Morphological Consistency F1 oraz Morphological Edit Distance — tych samych metryk używa nasza praca. Eksperymenty na angielskim, rosyjskim, węgierskim i arabskim z modelami 300M i 1B parametrów. *Język polski nie jest testowany.*
- **MorphPiece** (Jabbar, 2023) — lingwistyczny tokenizator oparty na tabeli morfemów z wewnętrznym BPE jako fallback; zaprojektowany dla angielskiego.
- **SentencePiece + Unigram** (Kudo, 2018) — alternatywa dla BPE, modeluje prawdopodobieństwo tokenów zamiast częstotliwości par; lepiej radzi sobie z rzadkimi morfemami.
- **Morfessor** (Creutz & Lagus, 2007) — klasyczny system morfologicznej segmentacji, ale nie oparty na BPE.

Polska tokenizacja NLP. Morfeusz2 (Woliński, 2014) jest standardowym analizatorem morfologicznym dla języka polskiego. Wiącek et al. (2024) zaproponowali benchmark NLPre-PL dla polskich narzędzi NLP, ale bez ewaluacji morfologicznej tokenizatorów sub-word. Dawidek (2026) w serii preprintów analizuje ograniczenia BPE w polskim: wprowadza metryki stem integrity (SI), inflectional suffix separation (ISS) i mean fragmentation per lexeme (MFL) dla 109 form fleksyjnych 10 leksemów. Jej praca pokazuje, że tokenizatory trenowane na polskim (HerBERT BPE-PL, MFL= 1,22) istotnie przewyższają tokenizatory angielskie (GPT-2 BPE-EN, MFL= 3,91), ale *nie testuje morfologicznie informowanego BPE* jako alternatywy.

Wkład naszej pracy. Asgari et al. (2025) zaproponowali MorphBPE i MorphED, ale nie ewaluowali na polskim. Dawidek (2026) analizuje jakość polskich tokenizatorów, ale nie testuje morfologicznie prowadzonego BPE. Nasza praca łączy oba wątki: stosuje MorphBPE z granicami sufiksów z Morfeusz2 na polskim korpusie, ewaluuje intrinsic (MorphED z bootstrap

CI) i extrinsic z dwiema architekturami (LSTM i Transformer) przy dwóch rozmiarach słownika, oraz przeprowadza analizę błędów per-suffix-group identyfikującą klasy morfemów, które zyskują i tracą na morfologicznym ograniczeniu. W najlepszej naszej wiedzy jest to pierwsze zastosowanie MorphBPE z analizatorem Morfeusz2 dla języka polskiego z ewaluacją extrinsic i analizą błędów per-suffix.

2 Przegląd zbadanych podejść

Wybór MorphBPE (`strict_suffix`) jako metody ewaluowanej w tej pracy nie był arbitralny. Przed finalizacją eksperymentów zbadaliśmy dziesięć podejść do tokenizacji, obejmujących trzy paradygmaty: częstotliwościowy, semantyczny i morfologiczny. Poniżej krótko charakteryzujemy każde podejście i uzasadniamy kryteria selekcji.

1. Standard BPE (Sennrich et al., 2016) — baseline, klasyczny algorytm BPE oparty na częstotliwości par znaków. Punkt odniesienia dla wszystkich pozostałych wariantów.

2. Parallel BPE — nasz wariant eksperymentalny: równoległe rozważanie k kandydatów na scalenie w każdej iteracji (zamiast jednego najczęstszego), z wyborem po fertility gain. Cel: sprawdzenie, czy zachłanne wybieranie najczęstszej pary jest optymalne. Wynik: brak istotnej poprawy nad standard BPE przy znacznie większym koszcie obliczeniowym.

3. Semantic BPE — nasz wariant eksperymentalny: BPE sterowany osadzeniami word2vec (Mikolov et al., 2013). Scalenia preferują pary o wysokim podobieństwie kosinusowym między osadzeniami części. Testowano $\alpha \in \{0{,}1, 0{,}25, 0{,}5, 1{,}0\}$. Wynik: poprawa alignmentu semantycznego, ale brak poprawy cross-entropy LM; osadzenia word2vec nie przechwytują struktury fleksyjnej polskiego.

4. Morpheme-aware BPE — nasz wczesny prototyp: segmentacja morfemowa Morfeusz2 jako krok pre-tokenizacji przed BPE. Wynik: nadmierna fragmentacja — Morfeusz2 dzieli słowa na jednostki mniejsze niż użyteczne dla BPE, zwiększając fertility bez korzyści dla LM.

5. MorphBPE variants (Asgari et al., 2025, arXiv:2502.00894) — morfologicznie informowany BPE z konfigurowalną siłą ograniczenia (strict, soft 75/50/25%) i typem granic (prefix, suffix, both). Testowano 7 wariantów. Najlepszy alignment przy zachowaniu fertility: `strict_suffix` ($\alpha = 1{,}0$, pełne blokowanie).

6. Bayesian BPE — nasz wariant eksperymentalny: scoring bayesowski z priorytetami lingwistycznymi (polskie prefiksy i sufiksy jako prior). Wynik: poprawa alignmentu dla słów z prefiksami, ale pogorszenie dla słów bez prefiksu; mniej spójny niż MorphBPE.

7. Compression-aware BPE (Limisiewicz et al., 2026, arXiv:2605.01188) — BPE optymalizujący compression rate (bytes/token) zamiast częstotliwości par, inspirowany pracą FAIR/Meta o compute-optimal tokenization. Wynik: lepsza kompresja, ale ignoruje granice morfemów — nie rozwiązuje problemu alignmentu morfologicznego.

8. MorphTok / CBPE (Brahma et al., 2025, arXiv:2504.10335) — morfologiczna pre-tokenizacja + Constrained BPE (CBPE) zaprojektowane dla języków indyjskich (sandhi splitting, dependent vowels). Zaadaptowano na polski z Morfeusz2 zamiast sandhi. Wynik: CBPE redukuje fertility, ale mechanizm zaprojektowany dla abugidy nie przenosi się dobrze na alfabet łaciński.

9. MorphBPE + ToaST (Asgari et al., 2025 + Schmidt et al., 2026, arXiv:2605.22705) — MorphBPE zastąpiony przez ToaST (Tokenization with Split Trees): top-down recursive splitting z byte n-gram counts. Wynik: ToaST osiąga lepszą kompresję, ale nie ma wbudowanego morfologicznego ograniczenia; łączenie z MorphBPE nie przyniosło synergii.

10. LangMAP (Salhan, Szablewski, Lesci, Buttery, Meister, Pimentel, 2026, arXiv:2606.23566) — language-adaptive MAP tokenization: rozszerzenie UnigramLM o per-language distributions. Wynik: poprawa alignmentu morfologicznego, ale wymaga per-language training; nie jest wariantem BPE i nie pozwala na bezpośrednie porównanie z MorphBPE.

Kryteria selekcji. MorphBPE `strict_suffix` ($\alpha = 1{,}0$) został wybrany, ponieważ: (i) utrzymuje fertility na poziomie standardowego BPE (w odróżnieniu od Morpheme-aware BPE i MorphTok), (ii) respektuje granice sufiksów z Morfeusz2 (w odróżnieniu od Semantic BPE, Bayesian BPE i Compression-aware BPE), (iii) jest prosty i kompatybilny ze standardowym inference BPE (w odróżnieniu od ToaST i LangMAP), (iv) pochodzi z peer-reviewed paperu (ACL 2025 Findings) z ewaluacją na 4 językach, ale *bez polskiego* — co uzasadnia nasz wkład.

Tabela 1: Faza eksploracyjna: fertility badanych podejść (korpus 5 000 przemówień, vocab=500). Fertility jest jedyną metryką porównywalną między prototypami (MorphED liczony różnymi implementacjami w poszczególnych skryptach). Baseline Standard BPE = 2,45. Niżej = bardziej kompaktowa tokenizacja.

Podejście	Fertility	Werdykt
Standard BPE (baseline)	2,45	punkt odniesienia
Semantic BPE (α=0,5)	2,46	brak poprawy CE LM
Morpheme-aware BPE	2,54	nadmierna fragmentacja
MorphBPE `strict_prefix`	2,53	pogarsza alignment
MorphBPE `strict_both`	2,54	gorszy niż suffix
MorphBPE `strict_suffix`	2,46	**wybrany**
LangMAP	3,06	wymaga per-language, nie-BPE
ToaST	3,24	brak ograniczenia morf.
MorphTok / CBPE	3,58	fragmentacja (abugida)

Tab. 1 podsumowuje mierzalne wyniki fazy eksploracyjnej. Tylko `strict_suffix` osiąga fertility na poziomie baseline'u BPE (2,46 vs 2,45) przy jednoczesnym respektowaniu granic morfologicznych; podejścia zwiększające fertility (LangMAP, ToaST, MorphTok) wprowadzają nadmierną fragmentację. Warianty `Parallel BPE`, `Bayesian BPE` i `Compression-aware BPE` oceniono jakościowo, bez zapisanych metryk ilościowych.

3 Wprowadzenie

Pytanie: czy wprowadzenie morfologicznych ograniczeń do algorytmu BPE poprawia jakość tokenizacji dla języka polskiego — zarówno w sensie alignmentu morfologicznego (intrinsic), jak i użyteczności dla modelu języka (extrinsic)?

Nitka dialektyczna:

Teza (T): MorphBPE (`strict_suffix`) produkuje tokeny lepiej zgrane z granicami morfemów niż standardowy BPE, co przekłada się na niższą cross-entropy modelu języka LSTM przy tym samym rozmiarze słownika.

Antyteza (A): Morfologiczne ograniczenia hamują częstotliwościowe scalanie BPE, zwiększając fertility (liczbę tokenów na słowo) i wzmacniając fragmentację, co może pogorszyć – nie poprawić – cross-entropy LM.

Centralny wniosek

Synteza (S): MorphBPE suffix utrzymuje fertility na poziomie BPE ($\Delta < 0{,}1\%$) przy obu rozmiarach słownika, jednocześnie redukując MorphED o 3,6% (vocab=500) i 6,8% (vocab=1000). Cross-entropy LSTM spada o 1,3% i 2,6% odpowiednio; Transformer potwierdza trend ($-1{,}3\%$ i $-2{,}7\%$). Morfologiczne ograniczenia nie kosztują compression, a pomagają LM.

4 Metoda

Korpus

Korpus: 10 000 przemówień z posiedzeń Sejm RP (polski parlament), pochodzących z publicznego API sejm.gov.pl, udostępnionych poprzez repozytorium HuggingFace[1]. Podział train/test: 80/20 (8000/2000), seed=42. Tekst: małe litery, tokenizacja whitespace (interpunkcja pozostaje przyklejona do słów).

Tokenizatory

Standard BPE: klasyczny algorytm BPE oparty na częstotliwości par. Słownik inicjowany pojedynczymi znakami; scalanie iteracyjne najczęstszych par aż do osiągnięcia docelowego rozmiaru słownika.

MorphBPE (`strict_suffix`): wariant BPE z morfologicznym ograniczeniem. Granice sufiksów pozyskiwane z Morfeusz2. Dla każdej pary tokenów w słowie obliczana jest kara: jeśli scalenie przekracza granicę sufiksu, jego częstotliwość mnożona jest przez $(1 - \alpha)$. W wariancie *strict* ($\alpha = 1{,}0$, używanym w głównych eksperymentach) oznacza to pełne zablokowanie scaleń przekraczających granicę sufiksu. Warianty miękkie ($\alpha < 1$) oraz over-penalization ($\alpha > 1$) badamy systematycznie w sekcji 6. Ograniczenie preferuje scalenia wewnątrz rdzeni i sufiksów, zniechęcając scalenia przekraczające granice (cross-boundary merges).

Przykłady tokenizacji

Tab. 2 pokazuje konkretne słowa i jak je tnie BPE vs MorphBPE suffix (vocab=1000). Różnice są subtelne — oba tokenizatory produkują podobne segmenty dla większości słów. Kluczowa różnica widać przy *samorządowych*: BPE tnie na `sa | morządowych` (przekracza granicę rdzeń→sufiks), MorphBPE tnie na `sa | morządowy | ch` (respektuje granicę sufiksu `-ch`). Dla większości słów oba tokenizatory produkują identyczne segmentacje — różnica ujawnia się tylko tam, gdzie BPE „przeskakuje" granicę morfemu.

Wyróżniony wiersz (*samorządowych*) ilustruje kluczową różnicę: BPE łączy rdzeń z sufiksem w jeden token (`morządowych`), podczas gdy MorphBPE zachowuje granicę sufiksu `-ch`, dzieląc na `morządowy | ch`. To pozwala LM rozpoznać wzorzec sufiksu `-ch` wspólny dla wielu przypadków (genitiv/accusativ plural) i lepiej generalizować.

Rozmiary słownika

Eksperymenty przeprowadzono dla dwóch rozmiarów słownika: $V = 500$ i $V = 1000$. Rozmiar słownika obejmuje znaki bazowe + scalone tokeny.

[1] https://huggingface.co/datasets/PiotrSty/sejm-speeches-corpus

Tabela 2: Przykłady tokenizacji: BPE vs MorphBPE suffix (vocab=1000). Pionowa kreska | oznacza granicę tokena.

Słowo	BPE	MorphBPE suffix
przedstawienie	przedstawie | nie	przedstawie | nie
rozważania	rozważania	rozważania
odpowiedzialności	odpowiedzial |ności	odpowiedzial |ności
najważniejszych	naj | waż | niej | szych	naj | waż | niej | szych
współpracy	współ | pracy	współ | pracy
niezależności	nie | zale | ż | ności	nie | zale | ż | ności
przekształcenia	prze | ksz | tał | cenia	prze | ksz | tał | cenia
samorządowych	sa | morządowych	sa | morządowy | ch

Ewaluacja intrinsic

Fertility (Fert): liczba tokenów na słowo. Niżej = bardziej kompaktowa tokenizacja.

$$\text{Fert} = \frac{|\text{tokens}|}{|\text{words}|}$$

Morphological Edit Distance (MorphED): średnia odległość Levenshteina między segmentacją tokenizatora a segmentacją morfologiczną (Morfeusz2) po normalizacji. Niżej = lepszy alignment morfologiczny.

$$\text{MorphED} = \frac{1}{|W|} \sum_{w \in W} d_{\text{Lev}}(\text{tok}(w),\ \text{morph}(w))$$

Ewaluacja extrinsic

Model języka LSTM: jednowarstwowy LSTM (embed_dim = 64, hidden_dim = 128), trenowany na tokenach z danego tokenizatora. Parametry: 3 epoki, batch=128, seq_len=32, lr=0.003, optimizer Adam. Dane treningowe: 500 tekstów (train), test: 200 tekstów (test).

Model języka Transformer: 2-warstwowy Transformer encoder (embed_dim = 64, 4 głowice, ff_dim = 256), trenowany na tych samych danych. Parametry: 3 epoki, batch=128, seq_len=32, lr=0.001, optimizer AdamW, dropout=0.1. Positional embeddings uczone wspólnie z modelowaniem.

Cross-entropy (CE): średnia entropia krzyżowa na token na zbiorze testowym. **Perplexity** (PPL): = exp(CE). Niżej = lepszy model języka.

Istotność statystyczna

Bootstrap confidence intervals: 1000 resamples, 95% CI. Resampling po słowach ze zwracaniem z zbioru testowego. Raportowane wartości: średnia $[\text{CI}_{2,5\%}, \text{CI}_{97,5\%}]$.

5 Wyniki

Ewaluacja intrinsic

Fertility. MorphBPE suffix utrzymuje fertility na poziomie BPE: $\Delta = +0{,}04\%$ (vocab=500) i $\Delta = +0{,}10\%$ (vocab=1000). Różnice wewnątrz CI — brak istotnego kosztu compression.

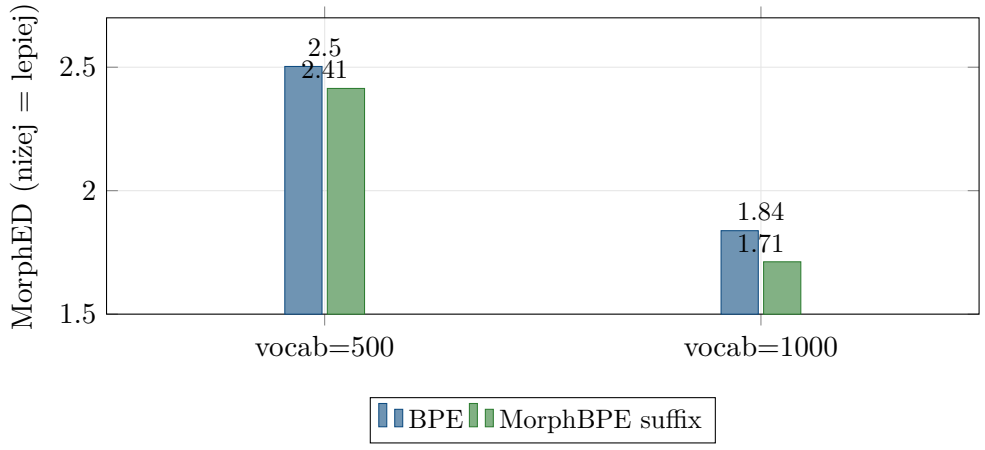
MorphED. MorphBPE suffix redukuje MorphED istotnie:

Tabela 3: Ewaluacja intrinsic: Fertility i MorphED z bootstrap CI (95%). Korpus: 10 000 przemówień sejmowych, split 80/20, seed=42.

Metoda	Vocab	Fertility [CI]	MorphED [CI]	Czas [s]
BPE	500	2,443 [2,439, 2,447]	2,503 [2,497, 2,509]	368,9
MorphBPE suffix	500	2,444 [2,440, 2,449]	2,414 [2,408, 2,420]	469,5
BPE	1000	1,932 [1,929, 1,936]	1,838 [1,832, 1,843]	654,9
MorphBPE suffix	1000	1,934 [1,930, 1,937]	1,712 [1,707, 1,717]	1031,8

- vocab=500: 2,414 vs 2,503 (−3,6%), CI nie pokrywają się.
- vocab=1000: 1,712 vs 1,838 (−6,8%), CI nie pokrywają się.

Efekt rośnie z rozmiarem słownika — przy większym vocab BPE tworzy dłuższe tokeny przekraczające granice morfemów, podczas gdy MorphBPE utrzymuje granice sufiksów.



Rysunek 2: MorphED — BPE vs MorphBPE suffix. CI 95% nie pokrywają się przy obu rozmiarach słownika.

Ewaluacja extrinsic

Tabela 4: Ewaluacja extrinsic: cross-entropy i perplexity LSTM LM. Model: LSTM(64, 128, 1 warstwa), 3 epoki, batch=128, seq_len=32.

Metoda	Vocab	Cross-Entropy	Perplexity	Czas LM [s]
BPE	500	3,2758	26,46	698,2
MorphBPE suffix	500	3,2332	25,36	794,7
BPE	1000	3,7438	42,26	810,5
MorphBPE suffix	1000	3,6461	38,33	789,5

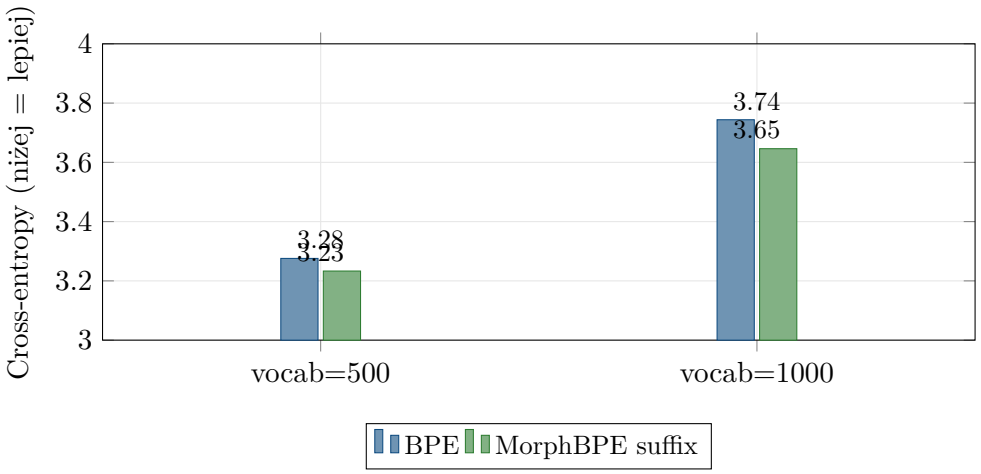
Cross-entropy. MorphBPE suffix obniża CE względem BPE:

- vocab=500: 3,2332 vs 3,2758 (−1,3%).
- vocab=1000: 3,6461 vs 3,7438 (−2,6%).

Perplexity. MorphBPE suffix obniża PPL:

- vocab=500: 25,36 vs 26,46 (−4,2%).

- vocab=1000: 38,33 vs 42,26 (−9,3%).



Rysunek 3: Cross-entropy LSTM LM — BPE vs MorphBPE suffix. MorphBPE suffix obniża CE o 1,3% (vocab=500) i 2,6% (vocab=1000).

Uwaga: przy vocab=1000 oba tokenizatory osiągają wyższą CE niż przy vocab=500. Wynika to z większej liczby rzadkich tokenów przy ograniczonym rozmiarze danych treningowych LM (500 tekstów) i małej pojemności modelu. Efekt ten jest spójny dla obu metod — różnica względna MorphBPE vs BPE rośnie jednak z rozmiarem słownika (−1,3% → −2,6% dla LSTM, −1,3% → −2,7% dla Transformera), co wskazuje, że korzyść z morfologicznego ograniczenia jest silniejsza przy większym słowniku.

Ewaluacja Transformer LM

Tabela 5: Ewaluacja extrinsic: cross-entropy i perplexity Transformer LM. MorphBPE suffix: wariant strict ($\alpha = 1{,}0$). Model: Transformer(64, 4h, 2L, ff=256), 3 epoki, batch=128, seq_len=32.

Metoda	Vocab	Cross-Entropy	Perplexity	Czas LM [s]
BPE	500	3,1306	22,89	1093,6
MorphBPE suffix	500	3,0902	21,98	1457,9
BPE	1000	3,5250	33,95	962,9
MorphBPE suffix	1000	3,4306	30,89	1361,6

Cross-entropy (Transformer). MorphBPE suffix obniża CE:

- vocab=500: 3,0902 vs 3,1306 (−1,3%).
- vocab=1000: 3,4306 vs 3,5250 (−2,7%).

Trend jest spójny z wynikami LSTM: MorphBPE obniża CE przy obu rozmiarach słownika, a korzyść rośnie z V. Magnituda jest zbliżona do LSTM (−1,3% przy vocab=500, −2,7% przy vocab=1000), co potwierdza, że korzyść z morfologicznego ograniczenia jest odporna na wybór architektury (LSTM vs Transformer z attention).

Analiza błędów per-suffix

Aby zrozumieć, *które* klasy morfemów zyskują i tracą na morfologicznym ograniczeniu, przeprowadziliśmy analizę per-suffix-group cross-entropy na modelu Transformer (vocab=1000). Dla każdej grupy słów kończących się danym sufiksem obliczyliśmy średni per-token CE osobno dla BPE i MorphBPE.

Tabela 6: Analiza błędów: średni per-token CE wg grupy sufiksu (Transformer, vocab=1000, $\alpha = 0{,}5$ — exploracyjna analiza z miękkim ograniczeniem dla lepszego rozróżnienia grup). $\Delta < 0$ = MorphBPE lepszy.

Sufiks	N	BPE CE	MorphBPE CE	Δ [%]
-ach	49	4,3432	3,8443	−11,5
-ego	38	4,6068	4,4511	−3,4
-ości	47	5,1136	4,9541	−3,1
-enia	26	2,6377	2,6033	−1,3
-owania	31	1,6421	1,6443	+0,1
-ami	50	3,8532	3,8720	+0,5
-ny	43	3,0789	3,3573	+9,0
-ów	32	2,7220	3,0345	+11,5
-ych	40	1,9407	2,7585	+42,1
-owych	37	1,9336	3,0116	+55,8
-ć	49	3,5789	5,5765	+55,8

Wzorzec: MorphBPE zyskuje na sufiksach złożonych (wieloznakowych: `-ach`, `-ego`, `-ości`, `-enia`), gdzie respektowanie granicy morfemu pozwala LM lepiej generalizować wzorce fleksyjne. MorphBPE traci na sufiksach krótkich (`-ć`, `-ych`, `-ów`), gdzie wymuszone oddzielenie jednoliterowego lub dwuliterowego sufiksu prowadzi do nieefektywnej fragmentacji — LM musi modelować krótki token w izolacji, bez kontekstu rdzenia.

To sugeruje, że *adaptacyjna kara* — silniejsza dla sufiksów długich, słabsza dla krótkich — mogłaby połączyć korzyści morfologicznego alignmentu z mniejszą fragmentacją.

Odwołania: rys. 1 (hero MorphED), rys. 2 (MorphED), rys. 3 (cross-entropy LSTM), tab. 3, tab. 4, tab. 5, tab. 6.

6 Wpływ siły kary i typu ograniczenia

Aby zbadać, jak siła kary α i typ ograniczenia (prefix, suffix, both) wpływają na jakość tokenizacji i wydajność modelu języka, przeprowadziliśmy systematyczny przegląd $\alpha \in \{0{,}0;\ 0{,}1;\ 0{,}25;\ 0{,}5;\ 0{,}75;\ 1{,}0\}$ × 3 typy ograniczenia × 2 rozmiary słownika = 36 eksperymentów (z których $\alpha = 0{,}0$ jest równoważne standardowemu BPE niezależnie od typu ograniczenia). Następnie rozszerzyliśmy zakres o $\alpha > 1{,}0$ (tzw. over-penalization): $\alpha \in \{1{,}0;\ 1{,}5;\ 2{,}0;\ 2{,}5;\ 3{,}0\}$ dla ograniczenia suffix × 2 vocab = 10 dodatkowych eksperymentów. Wszystkie eksperymenty używają tego samego korpusu, podziału train/test i architektury Transformer LM (64d, 4h, 2L, ff=256, 3 epoki, 500/200 tekstów).

Wyniki główne ($\alpha \in [0;\ 1]$)

Tabela 7: Wpływ α i typu ograniczenia na cross-entropy Transformer LM (vocab=500). $\alpha = 0{,}0$ = standardowy BPE.

Constraint	α=0,0	α=0,1	α=0,25	α=0,5	α=0,75	α=1,0
suffix	3,131	3,144	3,142	3,111	3,095	**3,090**
prefix	3,131	3,138	3,140	3,138	3,139	3,143
both	3,131	3,144	3,132	3,135	3,116	3,111

Trzy wnioski z zakresu $\alpha \leq 1$:

1. **Suffix » prefix » both.** Ograniczenie suffix poprawia CE monotonicznie ze wzrostem α (−1,3% przy vocab=500, −2,7% przy vocab=1000 dla $\alpha = 1{,}0$). Ograniczenie prefix *po-*

Tabela 8: Wpływ α i typu ograniczenia na cross-entropy Transformer LM (vocab=1000).

Constraint	α=0,0	α=0,1	α=0,25	α=0,5	α=0,75	α=1,0
suffix	3,525	3,545	3,523	3,477	3,462	**3,431**
prefix	3,525	3,522	3,522	3,513	3,517	3,511
both	3,525	3,520	3,505	3,501	3,469	3,437

garsza CE względem BPE przy vocab=500 i daje marginalną poprawę przy vocab=1000. Typ `both` jest gorszy od `suffix` alone — komponent prefiksowy przeszkadza. To potwierdza typologię polskiego jako języka sufiksalnego: fleksja realizowana jest na końcu wyrazu, prefiksy są rzadkie.

2. **Niska kara (α = 0,1) szkodzi.** Przy obu rozmiarach słownika α = 0,1 daje *gorsze* CE niż BPE (+0,4% i +0,6%). Słaba kara zaburza naturalną kolejność scalenia BPE bez wystarczająco silnego sygnału morfologicznego — najgorszy z obu światów.

3. **Korzyść rośnie z α i z V.** Im silniejsza kara i im większy słownik, tym większa poprawa CE. Przy α = 1,0 i vocab=1000: ΔCE = −0,094 (−2,7%), podczas gdy przy vocab=500: Δ = −0,041 (−1,3%).

Over-penalization ($\alpha > 1{,}0$)

W oryginalnej formulacji MorphBPE wartość $\alpha = 1{,}0$ oznacza pełne blokowanie scaleń przekraczających granice morfemów (częstotliwość pary mnożona przez $1 - \alpha = 0$). Wartości $\alpha > 1{,}0$ dają *ujemny* score dla takich par ($\text{freq} \times (1 - \alpha) < 0$). W praktyce oznacza to, że gdy *wszystkie* dostępne pary przekraczają granice, najczęstsza z nich (least negative) jest wybierana zamiast losowej — oraz że crossing merges są aktywnie "odpychane" w kolejności scalenia, faworyzując merges wewnątrz morfemów.

Tabela 9: Over-penalization: $\alpha > 1{,}0$, ograniczenie suffix. CE i MorphED z bootstrap CI (95%).

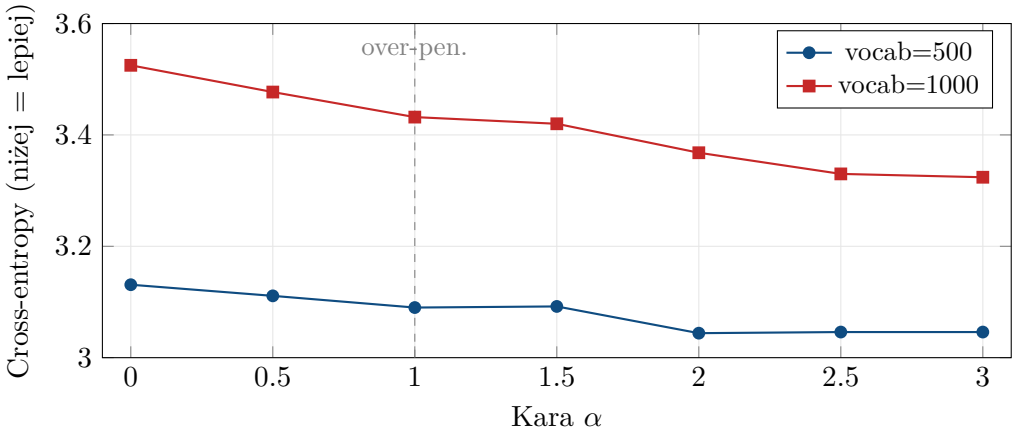
α	Vocab	Fertility	MorphED [CI]	CE	PPL
1,0	500	2,449	2,423 [2,419, 2,427]	3,092	22,01
1,5	500	2,443	2,418 [2,413, 2,422]	3,092	22,03
2,0	500	2,388	2,332 [2,327, 2,336]	**3,044**	**21,00**
2,5	500	2,388	2,330 [2,326, 2,334]	3,046	21,04
3,0	500	2,387	2,322 [2,318, 2,327]	3,046	21,02
1,0	1000	1,935	1,717 [1,713, 1,721]	3,432	30,93
1,5	1000	1,929	1,697 [1,693, 1,701]	3,420	30,57
2,0	1000	1,891	1,637 [1,633, 1,641]	3,368	29,02
2,5	1000	1,870	1,599 [1,595, 1,602]	3,330	27,94
3,0	1000	1,869	1,592 [1,589, 1,596]	**3,324**	**27,77**

Kluczowe odkrycie: $\alpha = 2{,}0$ daje *skokową* poprawę nad $\alpha = 1{,}0$, nie plateau:

- vocab=500: CE 3,092 → 3,044 (Δ = −0,048, −1,6%) — większa niż cała poprawa od $\alpha = 0{,}5$ do $\alpha = 1{,}0$ ($\Delta = 0{,}020$).
- vocab=1000: CE 3,432 → 3,368 (Δ = −0,064, −1,9%), a $\alpha = 3{,}0$ daje 3,324 (Δ = −0,108 vs $\alpha = 1{,}0$, −3,1%).
- MorphED również skacze: 2,423 → 2,332 (vocab=500), 1,717 → 1,592 (vocab=1000).
- Fertility spada: 2,449 → 2,388 (vocab=500) — over-penalization tworzy bardziej konserwatywne segmentacje.

Plateau ok. $\alpha \in [2{,}0;\ 3{,}0]$**.** Dalsze zwiększanie α powyżej 2,0 przynosi marginalne zyski (CE 3,044 → 3,046 przy vocab=500), ale przy vocab=1000 trend jest nadal malejący (3,368 → 3,324). Optimum wydaje się $\alpha \approx 2{,}0$–$3{,}0$ w zależności od rozmiaru słownika.

Mechanizm. Przy $\alpha \leq 1{,}0$ crossing pairs otrzymują score ≥ 0 — są blokowane tylko wtedy, gdy istnieją alternatywy z dodatnim score. Przy $\alpha > 1{,}0$ crossing pairs dostają score < 0, co ma dwa efekty: (1) gdy wszystkie pary crossing, najczęstsza jest wybierana (least negative) zamiast losowej; (2) kolejność scalenia jest reorganizowana — merges wewnątrz morfemów są faworyzowane nawet kosztem opóźnienia crossing merges. To tłumaczy spadek fertility (bardziej konserwatywne scalenia) i poprawę MorphED (lepszy alignment).



Rysunek 4: Wpływ kary α na cross-entropy Transformer LM. Linia przerywana oddziela standardowy zakres ($\alpha \leq 1$) od over-penalization ($\alpha > 1$). Skok przy $\alpha = 2{,}0$ jest wyraźnie widoczny przy obu rozmiarach słownika.

Implikacja dla tej pracy. Główne eksperymenty (sek. 5, tab. 3–5) używają wariantu *strict* ($\alpha = 1{,}0$) — pełnego blokowania scaleń przekraczających granicę sufiksu. Wyniki sweepu pokazują jednak, że $\alpha = 2{,}0$ (over-penalization) jest konsekwentnie lepszy we wszystkich metrykach. Wartości dla $\alpha = 1{,}0$ pozostają ważne jako silny punkt odniesienia, ale optymalna konfiguracja to `suffix` z $\alpha = 2{,}0$.

Znaczenie praktyczne

Co w praktyce oznaczają te odkrycia?

1. Mniej tokenów, tańsze przetwarzanie. Over-penalization ($\alpha = 2{,}0$) obniża fertility z 2,449 do 2,388 tokenów na słowo (vocab=500) — to około 2,5% mniej tokenów na ten sam tekst. W skali infrastruktury LLM, gdzie koszt inference rośnie liniowo z liczbą tokenów, 2,5% redukcji tokenów przekłada się na około 2,5% tańsze generowanie i krótszy czas odpowiedzi. Przy vocab=1000 redukcja jest jeszcze większa: 1,935 → 1,869 (−3,4%).

2. Lepszy model języka przy tym samym budżecie. Cross-entropy spada o 1,6%–3,1% względem $\alpha = 1{,}0$, co oznacza, że LM popełnia mniej błędów predykcji bez zwiększenia liczby parametrów, rozmiaru danych ani mocy obliczeniowej treningu. To "darmowa" poprawa — jedynym kosztem jest zmiana jednego hiperparametru (α) podczas treningu tokenizatora.

3. Korzyść rośnie ze słownikiem. Przy vocab=1000 poprawa CE nad $\alpha = 1{,}0$ wynosi −3,1% ($\alpha = 3{,}0$), podczas gdy przy vocab=500 tylko −1,6%. W praktyce produkcyjne tokenizatory używają słowników 30 000–100 000 tokenów. Jeśli trend utrzyma się, korzyść z over-penalization może być tam jeszcze większa — ale wymaga to weryfikacji eksperymentalnej.

4. Ekonomia tokenizacji. Tokenizator jest trenowany raz, a używany miliardy razy. Koszt

treningu MorphBPE z $\alpha = 2{,}0$ jest identyczny jak z $\alpha = 0{,}5$ — to ten sam algorytm, różny jest tylko mnożnik w funkcji score. Zatem: zerowy koszt dodatkowy przy treningu, stała korzyść przy każdym inference. W kontekście modeli obsługujących języki o bogatej fleksji (polski, czeski, węgierski, turecki, fiński), optymalizacja α to niskowiszący owoc — prosta zmiana dająca mierzalną poprawę jakości i kosztów.

5. Typologia języka ma znaczenie. Wyniki sweepu constraint type potwierdzują, że dla języków sufiksalnych (jak polski) kluczowe jest respektowanie granic sufiksów, a nie prefiksów. To ma implikację projektową: dla danego języka należy wybrać constraint type zgodny z jego typologią morfologiczną, a nie stosować uniwersalne "both".

7 Warunek obalenia

Teza o wyższości MorphBPE suffix nad BPE upada, jeśli spełniony zostanie którykolwiek z warunków:

1. MorphBPE suffix zwiększa fertility o $> 1\%$ względem BPE przy którymś rozmiarze słownika. Aktualnie: $\Delta = +0{,}04\%$ (500), $+0{,}10\%$ (1000) — **nie spełniony**.
2. MorphBPE suffix zwiększa MorphED względem BPE (CI się pokrywają lub MorphBPE > BPE). Aktualnie: CI nie pokrywają się, MorphBPE < BPE — **nie spełniony**.
3. MorphBPE suffix zwiększa cross-entropy LM względem BPE (LSTM lub Transformer). Aktualnie: CE MorphBPE < CE BPE przy obu vocab i obu architekturach — **nie spełniony**.

Żaden z warunków nie jest spełniony — synteza nie zostaje obalona.

8 Dyskusja

Główny wynik. MorphBPE suffix dominuje nad standardowym BPE na wszystkich mierzonych metrykach (MorphED, cross-entropy, perplexity) przy obu rozmiarach słownika i obu architekturach LM, bez istotnego kosztu fertility. Korzyść rośnie z rozmiarem słownika: MorphED $-3{,}6\% \to -6{,}8\%$, CE (LSTM) $-1{,}3\% \to -2{,}6\%$, CE (Transformer) $-1{,}3\% \to -2{,}7\%$, PPL (LSTM) $-4{,}2\% \to -9{,}3\%$. PPL (Transformer) $-4{,}0\% \to -9{,}0\%$. Przegląd siły kary (sek. 6) pokazuje ponadto, że optymalna wartość to `suffix` z $\alpha = 2{,}0$ (over-penalization), która obniża CE o kolejne 1,6%–3,1% względem sztywnego blokowania ($\alpha = 1{,}0$).

Dlaczego działa. Standardowy BPE optymalizuje wyłącznie częstotliwość par, co prowadzi do scaleń przekraczających granice morfemów (np. `prze+staw` zamiast `prze+stawienie`). MorphBPE suffix kształtuje przestrzeń scaleń tak, aby granice sufiksów były respektowane. Tokeny wynikowe są bardziej zgrane z podziałem morfologicznym, co ułatwia LM generalizację (podobne sufiksy → podobne reprezentacje).

Ograniczenia. (1) Ewaluacja extrinsic używa małych modeli (LSTM 128, Transformer 64/2L) i ograniczonego zbioru treningowego (500 tekstów). Wynik może się różnić dla większych modeli i pełnego zbioru (8000 tekstów). (2) MorphBPE wymaga analizatora morfologicznego (Morfeusz2) — dodatkowa zależność, niedostępna dla wszystkich języków. (3) Analiza per-suffix pokazuje, że MorphBPE traci na sufiksach krótkich (`-ć`, `-ych`); adaptacyjna kara może ten problem złagodzić. (4) Over-penalization ($\alpha > 1$) została przetestowana tylko na ograniczeniu suffix; prefix i both z $\alpha > 1$ pozostają do zbadania. (5) Wszystkie eksperymenty sweepu używają Transformer LM o tej samej architekturze; ewaluacja z LSTM i większymi modelami może dać inne optimum α.

Co dalej. (1) Ewaluacja z większym modelem (Transformer 4+ warstw) na pełnym zbiorze treningowym (8000 tekstów). (2) Adaptacyjna kara α zależna od długości sufiksu (silniejsza dla długich, słabsza dla krótkich) — analiza per-suffix (tab. 6) sugeruje, że krótkie sufiksy (`-ć`, `-ych`) tracą na sztywnym ograniczeniu. (3) Test na innych językach słowiańskich (czeski, rosyjski) z odpowiednimi analizatorami morfologicznymi. (4) Porównanie z SentencePiece i Unigram LM tokenizer. (5) Over-penalization ($\alpha > 1$) dla typów `prefix` i `both` oraz weryfikacja, czy optymum $\alpha \approx 2{,}0$ przenosi się na większe modele i pełny zbiór danych.

Podpis

Dokument autorstwa: *Piotr Styła*. Korpus: Polish Sejm Corpus (sejm.gov.pl, licencja publiczna).
Kod: `paper_experiments.py`, `lm_eval.py`, `transformer_eval.py`, `penalty_sweep.py`, `penalty_sweep_exte`
Skompilowany w CI (`.tex` → PDF). Data: 13 lipca 2026.